
[CS3704] Software Engineering

Dr. Chris Brown
Virginia Tech
9/10/2026

Requirements Analysis and Specification

Requirements Analysis

Use Cases

Requirements Specification

User Stories

Learning Outcomes

By the end of the course, students should be able to:

- **Understand software engineering processes, methods, and tools used in the software development life cycle (SDLC)**
- **Use techniques and processes to create and analyze requirements for an application**
- Use techniques and processes to design a software system
- Identify processes, methods, and tools related to phases of the SDLC
- Explain the differences between software engineering processes
- Discuss research questions and current topics related to software engineering
- Create and communicate about the requirements and design of a software application

Announcements

- **HW1 due *tomorrow* at 11:59pm**
- **PM1.1 due (Wednesday) 9/16 at 11:59pm**
 - **Problem Statement**
 - **Requirements Elicitation**
 - **Project Check-In Survey**
- **Survey for Greg Wilson guest lecture**

WEBINAR

THE NEW SHAPE OF CS CAREERS



DAN DUNLAP
Visiting Associate Professor
Department of Computer Science



MEG KURDZIOLEK
UX Lead
Intrinsic



MARK SHAPIRO
Director of Software Engineering
CoStar Group



DAVID SMITH IV
Assistant Professor
Department of Computer Science

Moderated by:



CHRISTINE JULIEN
Department Head
Department of Computer Science

Tuesday, Sept 22
7 p.m. EST
Virtual via Zoom or
On-Campus Watch Party

REGISTER HERE



COLLEGE OF ENGINEERING
COMPUTER SCIENCE
VIRGINIA TECH.

Requirements

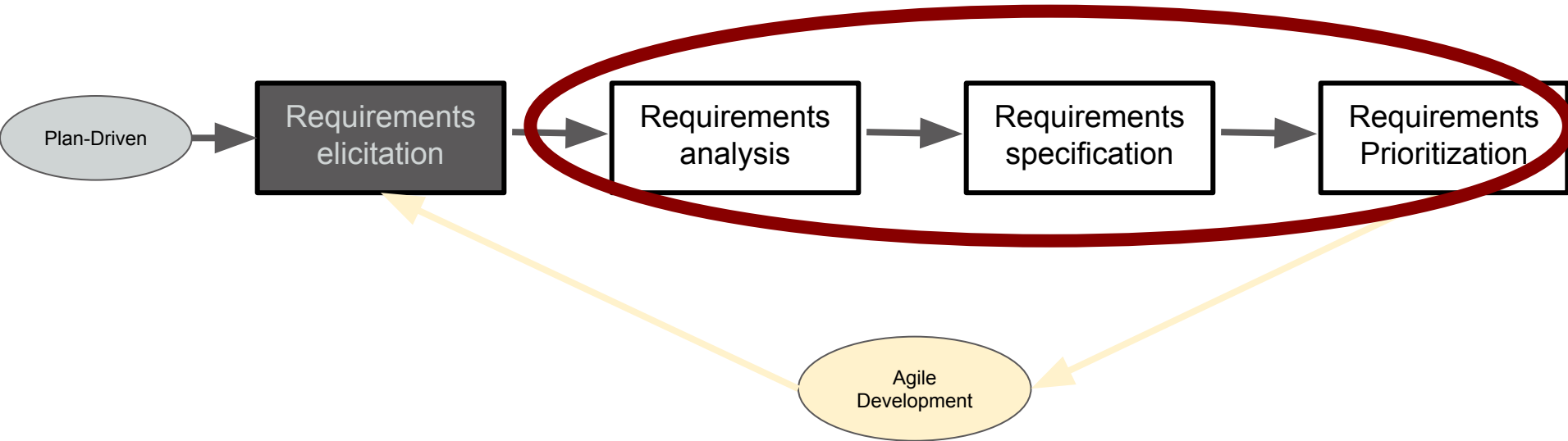
Rq

Goal: Understand customer requirements for the software system

- The *what* of the project
- Very difficult to “get right” the first time and evolve over the course of development
 - Remember the Top 3 reasons for project failure:
(2) Incomplete and (3) Changing Requirements
- *Software Artifacts:* use cases, diagrams, requirements specification docs, user stories...

Requirements Engineering

The process of defining, understanding, documenting, and maintaining requirements.



Running Lecture Example

Standup Bot: A software bot to automatically schedule standup meetings between teammates. 

- *Scrum master*: Professional to ensure scrum processes for development team

TODO: Complete a stand-up meeting!

- What I did.
- What I need to do next.
- What is blocking me.

Requirements Analysis

Requirements analysis involves *understanding* the needs and expectations of stakeholders for a system or project.

- Ensures developers and stakeholders are on the same page.
- Properly define requirements to reduce the risk of project failure or rework.
- Improves project planning to more accurately estimate time, cost, and resources.
- Enhances customer satisfaction by building software that meets the user's needs.

Artifacts

1. Use cases
2. Use case diagrams
3. Sequence diagrams

Use Cases

- Use case modeling is the most widely used approach for *requirements analysis*.
- **Use Case:** a written description of using the system to fulfill stakeholder goals.
 - A use case is a collection of related success and failure scenarios that describe the actor using a system to support a goal.
 - Useful for providing input into many subsequent software engineering activities and artifacts.

Use Case Terms

- **Actor:** something with the behavior
 - Person, computer system, organization
- **Scenario (use case instance):** a specific sequence of actions and interactions between actors and the system.
- **Preconditions:** What *must always* be true
 - Should be something noteworthy
 - “The user has power” ❌
 - “Scrum master is identified and authenticated” ✅
- **Postconditions:** What must be true on successful completion of the use case
 - Meeting is requested by authorized user, Date is correctly calculated, Bot has access to developer schedules, ...

Example: Stand-Up Bot

Use Case: Create a meeting

1 Preconditions

User must have google calendar api tokens in system.

2 Main Flow

User will request meeting and provide list of attendees [S1]. Bot will provide possible meeting times and user confirms [S2]. Bot creates meeting and posts link [S3].

3 Subflows

[S1] User provides /meeting command with @username,@username list.

[S2] Bot will return list of meeting times. User will confirm time.

[S3] Bot will create meeting and post link to google calendar event.

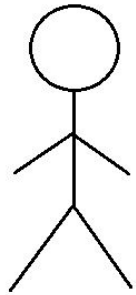
4 Alternative Flows

[E1] No team members are available.

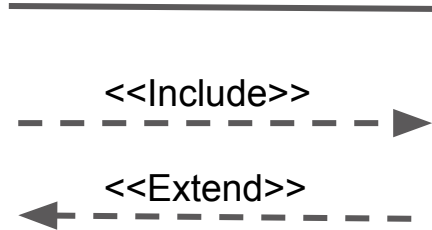
...

Use Case Diagrams

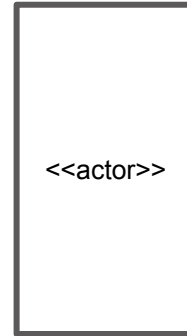
- Graphical depiction of use cases
 - Defines *actors'* interactions with the *system*



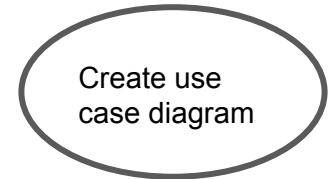
Actor



Interactions



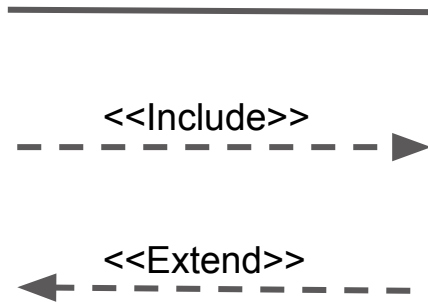
System



System Behavior

Use Case Diagrams (cont.)

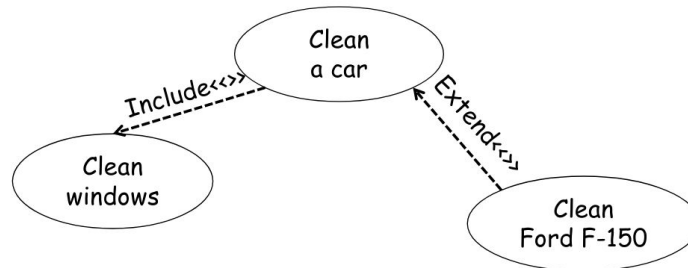
Interactions



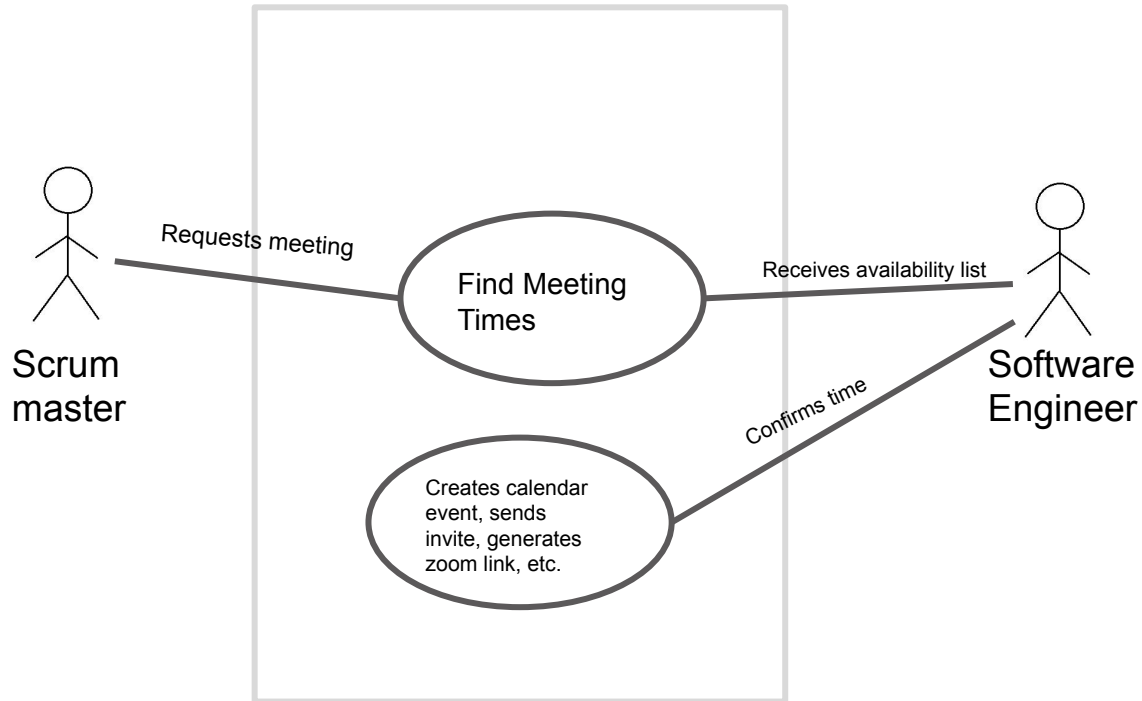
Association: relation between an actor and a use case

Includes dependency: a base use case includes a sub use case as component

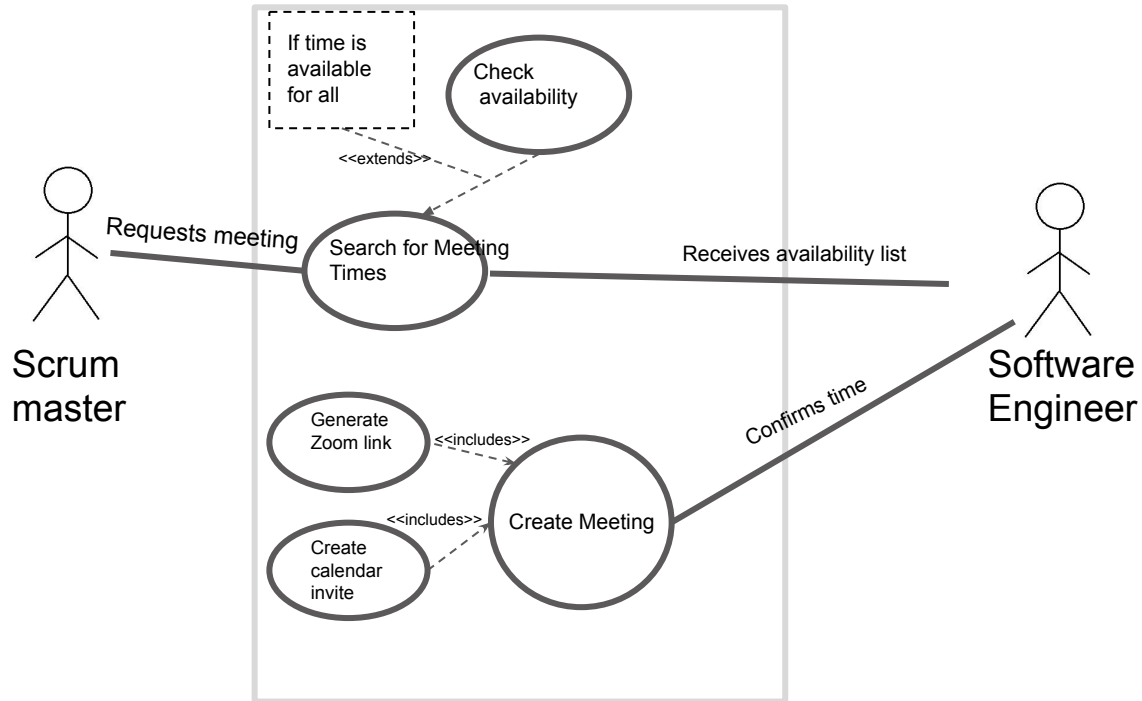
Extends dependency: a use case extends the behavior of a base use case.



Example: Stand-Up Bot

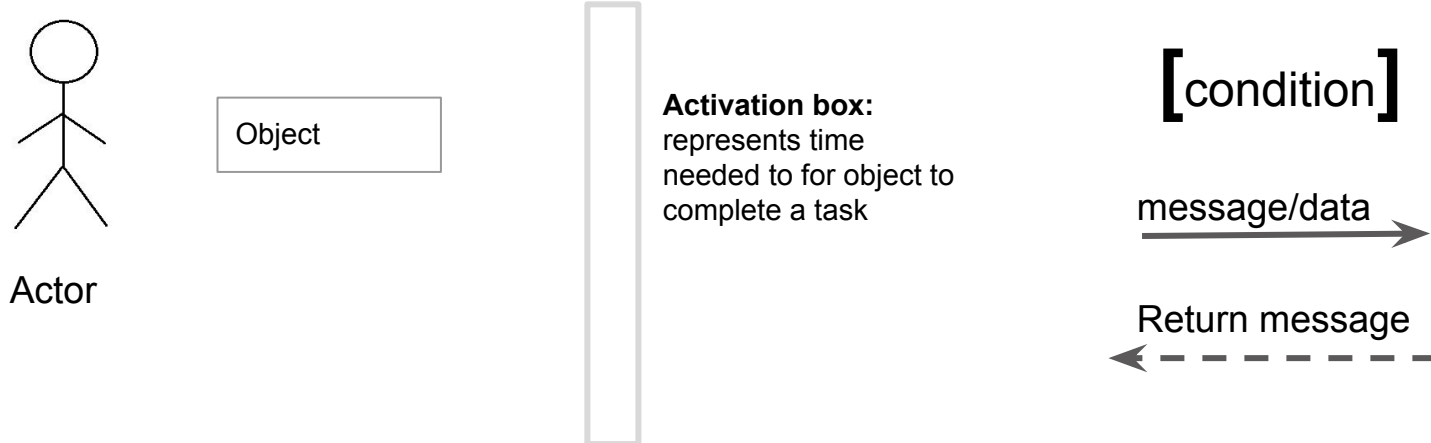


Example: Stand-Up Bot (cont.)

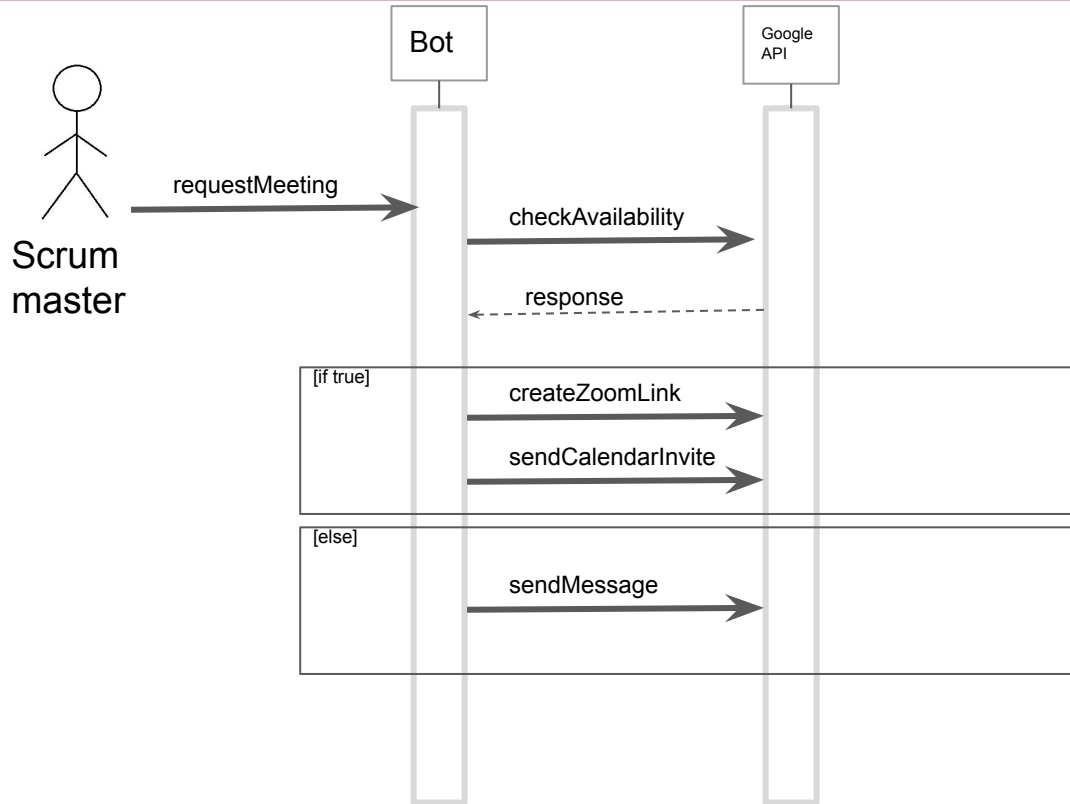


Sequence Diagrams

- A notation to illustrate, for a given use case, actor interactions and operations initiated by them.
 - Extends use case diagram to describe *how* and *in what order*.
 - Can support more advanced computational interactions (i.e., conditional statements, loops, etc.)



Example: Stand-Up Bot



Requirements Specification

Requirements specification involves *describing* what the system will do.

Artifacts:

1. Software Requirements Specification
2. User Stories

Application Domain

Machine Domain



Real-World Example

FBI Virtual Case File (VCF) Project

In 2000, the FBI began working on the [Virtual Case File \(VCF\) system](#) as part of a larger Trilogy Project to modernize their IT infrastructure. This project was contracted by Science Applications International Corp (SAIC) and intended to replace paper-based processes for managing investigations and intelligence. **However, by 2005, the project was abandoned after consuming over \$170 million.**



- *What went wrong?*

Real-World Example

FBI Virtual Case File (VCF) Project

- **What went wrong?** Poorly defined and specified requirements!
 - The developers/FBI lacked a clear understanding of processes and complex needs, which resulted in a system that was neither functional nor usable.
 - **Ambiguity:** Vague requirements, such as...
 - “Help FBI agents efficiently share data about cases in progress, especially terrorist investigations”
 - “Enable agents anywhere in the US quickly to search various documents and allow them to connect possible leads from different sources”
 - “Provide a case management system, an evidence management system, and a records management system to eliminate the need for FBI employees to scan hard-copy documents into computer files.”
 - **Over-specification:** “stop building a web front end to the FBI's existing systems and instead begin development of a new application to replace the old ACS”
 - **Wishful thinking:** “In January 2002, the FBI requested an additional \$70 million from Congress to accelerate the Trilogy project, but received \$78 million. SAIC agreed to deliver the VCF system in December 2003 rather than June 2004.”
 - But, “Trilogy's scope grew by about 80 percent [from the] initiation of the project.”
 - And many more...

Software Requirements Specification (SRS)

- A formal document that describes what a software system must do and how it behaves.
 - Introduction
 - Software Description
 - Functional Requirements
 - Non-functional Requirements
 - External Interface Requirements
 - Constraints and Limitations
- IEEE Standard

Iterative Requirements Specification

Iterative/agile Processes→little/no documentation!



User Stories

- Stakeholder-based scenarios with explicit acceptance criteria
 - Written from the point of view of a stakeholder.
 - Lightweight and iterative requirements specification.

“As a [stakeholder], I want to [action] so that [benefit].”

Given [initial context] → **When** [action taken]
→ **Then** [expected result]

Example User Story: Stand-Up Bot

As a Scrum Master, I want to find availability with a list of usernames so that I can schedule team daily stand-ups.

Acceptance criteria:

Given Scrum Master has admin permissions

When Scrum Master types team member usernames

And <User can perform multiple actions here>

Then System finds list of available times between users

Req. Analysis vs. Specification

Analysis leads to Specification – they are not the same!

“A user story is to a use case as a gazelle is to a gazebo”

[Cockburn]

- They sound similar, but are very different.

Requirements Prioritization

Prioritization is necessary because all software projects time, budget, and scope constraints.

Artifacts

1. Backlog

Review: Sprint Planning

- Meeting to review and determine tasks to be completed in the upcoming sprint
 - Entire team (Scrum master, manager, developers, etc.)
- Divide work between team members
 - How do you divide work between software engineers? [Wilson]
 - **Feature decomposition**- by feature aspects
 - **Modular decomposition**- by module (i.e. parts of code)
 - **Functional decomposition**- by set of tasks or skill
 - **Rotating decomposition**- by function, but swapping periodically
 - **Chaotic decomposition**- random
- Project Estimation (*Planning Poker*)

Sprint Planning (cont.)

Goals:

1. Define the outcomes that matter most.
2. Select *product backlog* items based on function points, capacity, stakeholders, risk...
 - **Backlog:** a list of evolving product needs
3. Create a *sprint backlog* and agree on a definition of *done* (i.e., acceptance criteria).

Prioritization Methods

- Planning Poker 
- Stakeholder Ranking
 - Clients classify features as basic needs, then negotiate scope → compare business value with implementation costs, risk, etc.
 - Examples:
 - Business/Requirements Analyst
 - Gap Analysis
 - Community Feedback

Business/Requirements Analysts

What do requirements analysts do?

- Provide a starting point
 - Some notion that there is a “problem” that needs solving
 - e.g. dissatisfaction with the current state of affairs
 - e.g. a new business opportunity
 - e.g. a potential saving of cost, time, resource usage, etc.
- The requirements analyst must:
 - Identify the “problem”/”opportunity”
 - Which problem needs to be solved? (identify problem Boundaries)
 - Where is the problem? (understand the Context/Problem Domain)
 - Whose problem is it? (identify Stakeholders)
 - Why does it need solving? (identify the stakeholders’ Goals)
 - How might a software system help? (collect some Scenarios)
 - When does it need solving? (identify Development Constraints)
 - What might prevent us solving it? (identify Feasibility and Risk)
 - and become an “expert” in the problem domain

Gap Analysis

- Evaluate the gaps in a product performance
 - Usually after a working product is available
 - Used by business analysts to present current state and target state of product

Gap Analysis Template				
	Current State (FROM)	Desired Future State (TO)	GAPS	Remedies
Innovation	We are not currently known for innovation, however our software does contain a couple of unique features.	To be recognised as one of the most innovative platforms in the industry.	40% Only 10% of our developer resources are working on new features. At least 50% of our developer resources working on new features.	Hire an additional 6 developers
Item 2				
Item 3				
Item 4				
Item 5				
Item 6				
Item 7				
Item 8				
Item 9				
Item 10				

Community Feedback

- Insights from users on most-desired features.
 - Ex) GitHub Issues and Discussions

The screenshot shows the PyGithub Discussions page. The top navigation bar includes links for Code, Issues (273), Pull requests (104), Discussions (active), Actions, Projects, Security, and Insights. A search bar is present with the text 'Type / to search'. Below the navigation bar, the search filters are set to 'is:open category:Ideas'. The page is titled 'Ideas' with the subtitle 'Share ideas for new features'. On the left, there are categories: General, Ideas (selected), Polls, Q&A, and Show and tell. A 'Most helpful' section is also visible. The main content area lists several ideas with their titles, authors, start dates, and the number of replies.

Upvotes	Idea Title	Author	Start Date	Replies
1	Automatically look for access tokens in standard places	gmishkin	started on Sep 13, 2024 in Ideas	4
1	Response caching	bobneuman	started on Jun 28, 2024 in Ideas	1
2	Do you have plans to add workflows features?	LuisSFGH	started on Sep 16, 2021 in Ideas	1
1	Top repo contributors	sr-murthy	started on May 28, 2022 in Ideas	0
2	Dedicated builder object for review comments?	xmo-odoo	started on Feb 11, 2022 in Ideas	0
1	Drop .pyi?	scoates	started on Jul 12, 2021 in Ideas	1

Putting it All Together

Use Case

Goal: create a meeting

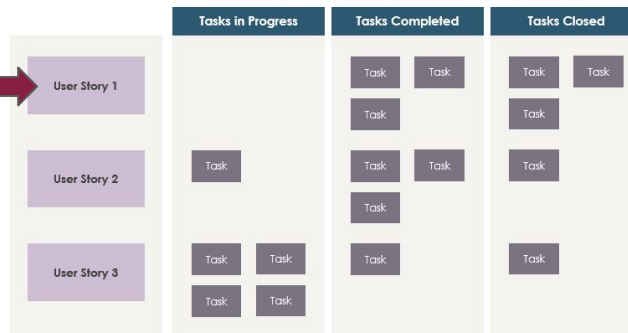
Preconditions: user is authenticated and calendar access is available.

Main flow: request attendees → check availability → show options → confirm time → create event → send invite.

Alternative: no common time is available.

User Story

As a Scrum Master, I want to find availability for a list of teammates so that I can schedule a daily stand-up meeting to track team progress.



Testable Req.

AC: Given an authorized Scrum Master and valid attendee calendars;
When the Scrum Master requests availability;
Then the system displays one or more valid time options or clearly reports that none exist.

Class Activity

Hold a Sprint Planning Meeting:

Based on your initial ideas for Project 1:

1. Create a list of general requirements/functionality for your system (can be formal use cases, user stories, or an informal list of ideas—ideally from requirements elicitation results!).
2. Use the MoSCoW method (see below) to rank the initial lists



Task	Name Role: Responsible	Name Role: Accountable	Name Role: Consulted	Name Role: Informed
Task 1: Task name				
Task 2: Task name				
Task 3: Task name				

3. Break requirements into subtasks and create a backlog using GitHub Projects. For the highest priority task(s), estimate the number of function points and start a RACI matrix (see above).
 4. Submit backlog screenshot and RACI matrix on Canvas (w/ names and PIDs of team here!).
- Due:** before the end of class today (submit what you complete, graded for attendance)

Next Class...

- **Software Architecture and Design**

Announcements

- **HW1 due *tomorrow* at 11:59pm**
- **PM1.1 due (Wednesday) 9/16 at 11:59pm**

References

- Asma Yamani, Malak Baslyman, and Moataz Ahmed. "Leveraging LLMs for User Stories in AI Systems: UStAI Dataset." In Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering, pp. 21-30. 2025.
- Aaron Conrardy. "From Image to UML: First Results of Image Based Class Diagram Generation Using LLMs". Modeling Languages. <https://modeling-languages.com/image-to-uml-with-llm/>
- Priya Pedamkar. "Types of UML Diagrams". <https://www.educba.com/types-of-uml-diagrams/>
- Zheyang Zhang, Maruf Rayhan, Tomas Herda, Manuel Goisau, and Pekka Abrahamsson. "LLM-Based Agents for Automating the Enhancement of User Story Quality: An Early Report." In International Conference on Agile Software Development, 2024.
- Petre, Marian. "UML in practice." In 2013 35th international conference on software engineering (icse), pp. 722-731. IEEE, 2013.
- Steve Easterbrook (University of Toronto)
- Sarah Heckman (NC State)
- Na Meng and Barbara Ryder (VT)